

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

***CO5:** Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting
(BL 3)*

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I **THAMIZHARASAN S (192424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
THAMIZHARASAN S

Annexure A

Short Answer Test

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.

ANSWER:

Pseudocode: Dataset Reading and Preprocessing

Input: Dataset file input.csv

Output: Cleaned dataset cleaned_dataset.csv

ALGORITHM- Preprocess_Dataset

INPUT:

FileName ← "input.csv"

OUTPUT:

Cleaned dataset ← "cleaned_dataset.csv"

BEGIN

1. READ DATASET

DataFrame ← ReadCSV(FileName)

2. CREATE INDEX

DataFrame ← ResetIndex(DataFrame)

Assign unique index to every record

3. REMOVE DUPLICATE RECORDS

DataFrame ← RemoveDuplicates(DataFrame)

// Keeps only unique rows

4. HANDLE MISSING VALUES

```
FOR each column in DataFrame DO
  IF column is numerical THEN
    Replace missing values with column median
  ELSE IF column is categorical THEN
    Replace missing values with column mode
  END IF
END FOR
```

5. FILTER INVALID ENTRIES

```
FOR each row in DataFrame DO
  IF row contains invalid values OR
    numerical value is outside allowed range THEN
    Remove row
  END IF
END FOR
```

6. NORMALIZE NUMERICAL COLUMNS

```
NumericalColumns ← Select numerical columns

FOR each column C in NumericalColumns DO
  MinValue ← Minimum(C)
  MaxValue ← Maximum(C)

  FOR each value x in C DO
     $x \leftarrow (x - \text{MinValue}) / (\text{MaxValue} - \text{MinValue})$ 
  END FOR
END FOR
```

7. STANDARDIZE CATEGORICAL VALUES

```
CategoricalColumns ← Select categorical columns

FOR each categorical column C DO
```

Convert values to lowercase
Remove leading/trailing spaces
Replace inconsistent labels with standard labels

Example:

"Male", "MALE", " male " → "male"

END FOR

8. FILTER USING DATAFRAME OPERATIONS

```
DataFrame ← Filter(DataFrame, valid conditions)  
// Retain only records satisfying required conditions
```

9. OPTIMIZE DATA

Remove unnecessary columns
Convert repeated categorical values to categorical data type
Use vectorized array/DataFrame operations instead of
processing rows individually wherever possible

10. RESET INDEX

```
DataFrame ← ResetIndex(DataFrame)
```

11. SAVE CLEANED DATASET

Write DataFrame to "cleaned_dataset.csv"

12. RETURN OUTPUT

RETURN DataFrame

END

Processing Flow

Input File → Read DataFrame → Indexing → Remove Duplicates → Handle Missing Values
→ Filter Invalid Entries → Normalize Numerical Data → Standardize Categorical Data →
Reset Index → Save Clean Dataset

Computational Efficiency

- Use vectorized DataFrame/array operations instead of unnecessary loops.
- Calculate median, mode, minimum, and maximum once per column.
- Remove duplicates before expensive preprocessing.
- Filter invalid records early to reduce the dataset size.
- Use appropriate data types for numerical and categorical columns.
- The cleaned dataset is finally saved as a new file, preserving the original dataset.

2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.

ANSWER:

Pseudocode: Statistical Measures for a Large Dataset

Input: Large numerical dataset D

Output: Mean, Median, Variance, and Standard Deviation

1. ALGORITHM CalculateStatistics

INPUT:

D ← Large numerical dataset
 LowerLimit, UpperLimit ← Valid range for data
 OutlierMethod ← "IQR"

OUTPUT:

Mean
 Median
 Variance
 StandardDeviation

BEGIN

1. LOAD DATASET

D ← ReadDataset("data.csv")

2. VALIDATE DATASET

IF D does not exist THEN

 Display "Dataset not found"

 STOP

END IF

IF D is empty THEN

 Display "Error: Dataset is empty"

 STOP

END IF

3. CONVERT TO ARRAY

A $\leftarrow$ ConvertToNumericArray(D)

4. REMOVE INVALID VALUES

A $\leftarrow$ Filter(A, value is NOT NULL)

A $\leftarrow$ Filter(A, value is NOT NaN)

A $\leftarrow$ Filter(A, value is finite)

5. CHECK AFTER FILTERING

IF Length(A) = 0 THEN

 Display "Error: No valid numerical values"

 STOP

END IF

6. APPLY RANGE FILTERING

ValidData $\leftarrow$ Filter(A,
 value $\geq$ LowerLimit AND
 value $\leq$ UpperLimit)

7. CHECK VALID DATA

IF Length(ValidData) = 0 THEN

 Display "Error: No values within valid range"

```
    STOP
END IF
```

8. DETECT EXTREME OUTLIERS

```
Sort ValidData
```

```
Q1 ← Calculate25thPercentile(ValidData)
```

```
Q3 ← Calculate75thPercentile(ValidData)
```

```
IQR ← Q3 - Q1
```

```
LowerBound ← Q1 - 1.5 × IQR
```

```
UpperBound ← Q3 + 1.5 × IQR
```

```
FilteredData ← Filter(ValidData,  
    value >= LowerBound AND  
    value <= UpperBound)
```

9. HANDLE OUTLIER EDGE CASE

```
IF Length(FilteredData) = 0 THEN
```

```
    Display "Warning: All values identified as outliers"
```

```
    FilteredData ← ValidData
```

```
END IF
```

10. CALCULATE NUMBER OF VALUES

```
N ← Length(FilteredData)
```

11. CALCULATE MEAN USING ARRAY AGGREGATION

```
Sum ← SumArray(FilteredData)
```

```
Mean ← Sum / N
```

12. CALCULATE MEDIAN

```
SortedData ← Sort(FilteredData)
```

IF N is odd THEN

 Median $\leftarrow$ SortedData[(N + 1) / 2]

ELSE

 Middle1 $\leftarrow$ SortedData[N / 2]

 Middle2 $\leftarrow$ SortedData[(N / 2) + 1]

 Median $\leftarrow$ (Middle1 + Middle2) / 2

END IF

13. CALCULATE VARIANCE

SquaredDifferences $\leftarrow$ (FilteredData - Mean)²

Variance $\leftarrow$ SumArray(SquaredDifferences) / N

14. CALCULATE STANDARD DEVIATION

StandardDeviation $\leftarrow$ SquareRoot(Variance)

15. RETURN RESULTS

Display "Number of valid values:", N

Display "Mean:", Mean

Display "Median:", Median

Display "Variance:", Variance

Display "Standard Deviation:", StandardDeviation

END

genui{"learning_viz":{"type_id":"STANDARD_DEVIATION"}}

2. Array-Based Operations

For a large dataset, operations should preferably be **vectorized** rather than processing every value individually.

Mean = SUM(A) / N

DifferenceArray = A - Mean

$\text{SquaredDifferenceArray} = \text{DifferenceArray} \times \text{DifferenceArray}$

$\text{Variance} = \text{SUM}(\text{SquaredDifferenceArray}) / N$

$\text{StandardDeviation} = \text{SQRT}(\text{Variance})$

This allows array libraries such as NumPy or DataFrame libraries such as pandas to perform calculations efficiently.

3. Handling Important Edge Cases

Edge Case	Action
Empty dataset	Stop and display an error
All values are missing	Stop after filtering
NaN/infinite values	Remove before calculation
Values outside valid range	Exclude using filtering
Extreme outliers	Detect using IQR
All values are outliers	Retain original valid data with warning
Single valid value	Mean = Median = value; variance = 0
Duplicate values	Keep them unless the application requires removal
Very large values	Use suitable numerical data types to reduce overflow risk

4. Computational Efficiency

For a large dataset:

1. Use **vectorized array operations** for sum, subtraction, multiplication, and filtering.
2. **Avoid unnecessary loops** over individual elements.
3. Filter invalid values **before** statistical calculations.
4. Use aggregation functions such as SUM, COUNT, and percentile operations.

5. Sorting is required for the median and IQR, so it is generally the most expensive step.
6. For extremely large or streaming datasets, mean and variance can be calculated incrementally without storing the entire dataset.

Final Output:

Number of valid values = N

Mean = Mean

Median = Median

Variance = Variance

Standard Deviation = StandardDeviation

3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.

ANSWER:

Pseudocode: Combining and Merging Multiple DataFrames

Input: Two or more DataFrames DF1, DF2, ..., DF_n containing related data

Output: A single validated, consistent, and sorted MergedDF

ALGORITHM Combine_DataFrames

INPUT:

DF1, DF2, ..., DF_n

CommonKey ← Common column used for merging

OUTPUT:

MergedDF ← Validated combined DataFrame

BEGIN

1. LOAD DATAFRAMES

DF1 ← ReadData("dataset1.csv")

DF2 ← ReadData("dataset2.csv")

IF more DataFrames exist THEN

 Load DF3, DF4, ..., DFn

END IF

2. VALIDATE INPUT DATAFRAMES

FOR each DataFrame DF DO

 IF DF is empty THEN

 Display "Warning: DataFrame is empty"

 END IF

 IF CommonKey does not exist in DF THEN

 Display "Error: Common key missing"

 STOP

 END IF

 Check data types of CommonKey

 IF data type of CommonKey is inconsistent THEN

 Convert CommonKey to a common data type

 END IF

END FOR

3. CLEAN THE COMMON KEY

FOR each DataFrame DF DO

 Remove leading and trailing spaces from CommonKey

 Remove or flag NULL values in CommonKey

Standardize the format of CommonKey

END FOR

4. ALIGN INDICES

FOR each DataFrame DF DO

 ResetIndex(DF)

END FOR

// Ensures that each DataFrame has a consistent index

5. REMOVE DUPLICATES

FOR each DataFrame DF DO

 Identify duplicate records using CommonKey

 IF duplicate records exist THEN

 Keep the required record

 Remove unnecessary duplicates

 END IF

END FOR

6. MERGE FIRST TWO DATAFRAMES

```
MergedDF ← Merge(  
    DF1,  
    DF2,  
    on = CommonKey,  
    method = "inner"  
)
```

// Inner merge keeps records with matching keys

7. MERGE REMAINING DATAFRAMES

FOR each DataFrame DFi from DF3 to DFn DO

```
MergedDF ← Merge(  
    MergedDF,  
    DFi,  
    on = CommonKey,  
    method = "inner"  
)
```

END FOR

8. CHECK FOR DUPLICATES AFTER MERGING

DuplicateCount ← CountDuplicates(MergedDF)

IF DuplicateCount > 0 THEN

```
MergedDF ← RemoveDuplicates(MergedDF)
```

END IF

9. CHECK MISSING VALUES

MissingCount ← CountMissingValues(MergedDF)

IF MissingCount > 0 THEN

Identify columns containing missing values

IF missing value can be replaced THEN

Replace using appropriate value

// Mean, median, mode, or predefined value

ELSE

Flag the affected records

END IF

END IF

10. VALIDATE KEY INTEGRITY

IF CommonKey contains NULL values THEN

 Display "Error: Invalid key values"

 STOP

END IF

IF CommonKey contains unexpected duplicates THEN

 Display "Warning: Duplicate keys detected"

END IF

11. VALIDATE RECORD COMPLETENESS

Compare number of records before and after merging

IF required records are missing THEN

 Display "Warning: Some records were not matched"

 Identify unmatched keys

END IF

12. GROUP DATA

```
GroupedData ← GroupBy(  
    MergedDF,  
    CommonKey  
)
```

FOR each group DO

 Calculate required aggregate values

 // Example: SUM, COUNT, MEAN

END FOR

13. SORT DATA

```
MergedDF ← Sort(  
    MergedDF,  
    CommonKey  
)
```

```
MergedDF,  
by = CommonKey,  
ascending = TRUE  
)
```

14. FINAL INTEGRITY CHECK

Check:

- CommonKey exists
- No invalid key values
- No unintended duplicates
- Required columns are present
- Data types are consistent
- Required records are complete

IF all checks pass THEN

Display "Merge successful"

ELSE

Display "Merge failed validation"

STOP

END IF

15. SAVE OUTPUT

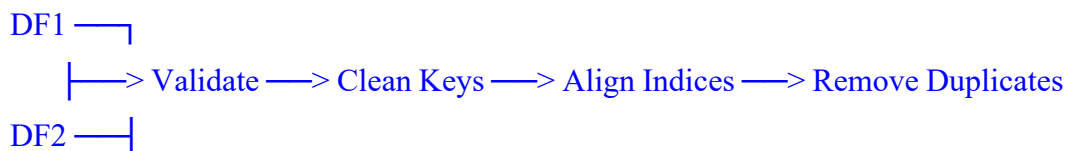
Write MergedDF to "merged_dataset.csv"

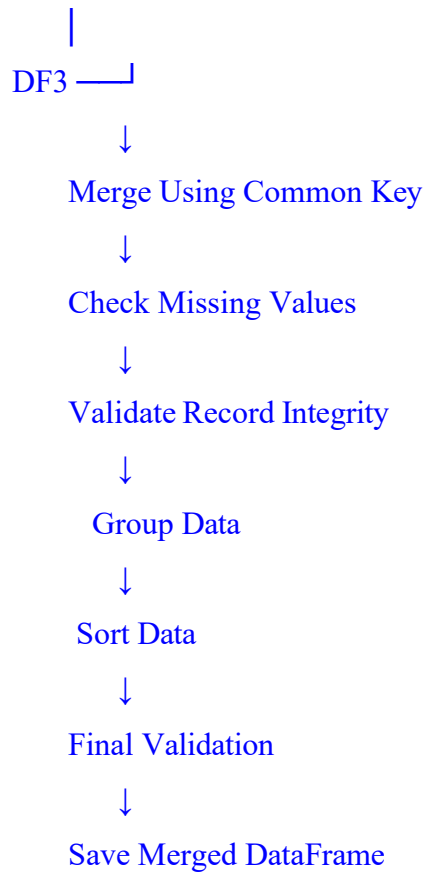
16. RETURN OUTPUT

RETURN MergedDF

END

Data Processing Flow





Example

Suppose:

DF1 – Student Details

Student_ID	Name	Department
101	Arun	ECE
102	Ravi	CSE
103	Priya	ECE

DF2 – Student Marks

Student_ID	Marks
101	85

102	90
103	78

Using Student_ID as the common key:

MergedDF = Merge(DF1, DF2, on = "Student_ID")

Result:

Student_ID	Name	Department	Marks
101	Arun	ECE	85
102	Ravi	CSE	90
103	Priya	ECE	78

Integrity Constraints

The algorithm maintains:

- **Consistency:** Common-key data types and formats are standardized.
- **Completeness:** Missing or unmatched records are identified.
- **Uniqueness:** Unintended duplicate records are removed.
- **Referential integrity:** Every merged record must have a valid common key.
- **Correct ordering:** Final records are sorted using the common key.
- **Validation:** The final DataFrame is checked before being saved.

This approach is efficient because DataFrame-level **merge, filter, group, sort, and aggregation operations** are used instead of unnecessary row-by-row processing.

4.Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.

ANSWER:

Pseudocode: Filtering and Sorting a Dataset Using Multiple Conditions

Input: Dataset D containing numerical and categorical columns

Output: Filtered and sorted dataset Result

Algorithm

ALGORITHM Filter_And_Sort_Dataset

INPUT:

D $\leftarrow$ Input dataset
MinValue, MaxValue $\leftarrow$ Required numerical range
ValidCategories $\leftarrow$ List of accepted categories
SortColumn $\leftarrow$ Column used for sorting
SortOrder $\leftarrow$ ASCENDING or DESCENDING

OUTPUT:

Result $\leftarrow$ Filtered and sorted dataset

BEGIN

1. LOAD DATASET

D $\leftarrow$ ReadDataset("input.csv")

2. VALIDATE DATASET

IF D is empty THEN
 Display "Error: Dataset is empty"
 STOP
END IF

3. CREATE INDEX

D $\leftarrow$ SetIndex(D, "ID")

4. CHECK REQUIRED COLUMNS

RequiredColumns $\leftarrow$ ["ID", "Value", "Category"]

FOR each Column in RequiredColumns DO

```
    IF Column does not exist in D THEN
        Display "Error: Required column missing"
        STOP
    END IF
END FOR
```

5. HANDLE INVALID VALUES

```
D ← RemoveRowsWhere(
    Value is NULL OR
    Value is NaN OR
    Category is NULL
)
```

6. DEFINE FILTER CONDITIONS

```
Condition1 ← (D["Value"] >= MinValue)
```

```
Condition2 ← (D["Value"] <= MaxValue)
```

```
Condition3 ← (D["Category"] IN ValidCategories)
```

7. COMBINE CONDITIONS

```
FinalCondition ← Condition1
                AND Condition2
                AND Condition3
```

8. FILTER DATASET

```
Result ← D[FinalCondition]
```

9. VALIDATE FILTERED RESULTS

```
FOR each record R in Result DO
```

```
IF R["Value"] < MinValue OR  
  R["Value"] > MaxValue THEN  
  Remove R  
END IF
```

```
IF R["Category"] NOT IN ValidCategories THEN  
  Remove R  
END IF
```

```
END FOR
```

10. CHECK EMPTY RESULT

```
IF Result is empty THEN  
  Display "No records satisfy the filtering conditions"  
  STOP  
END IF
```

11. SORT FILTERED RESULTS

```
Result ← Sort(  
  Result,  
  by = SortColumn,  
  order = SortOrder,  
  algorithm = "efficient stable sort"  
)
```

12. RESET INDEX

```
Result ← ResetIndex(Result)
```

13. FINAL VALIDATION

```
IF every record satisfies FinalCondition THEN
    Display "Filtering constraints satisfied"
ELSE
    Display "Validation failed"
    STOP
END IF
```

14. SAVE OUTPUT

```
Write Result to "filtered_sorted_dataset.csv"
```

15. RETURN OUTPUT

```
RETURN Result
```

END

Example of Multiple Conditions

Suppose the dataset contains:

- Age
- Category
- Score

Required conditions:

Age ≥ 18

AND

Age ≤ 30

AND

Category $\in \{\text{"ECE"}, \text{"CSE"}\}$

AND

Score ≥ 60

The logical filtering operation is:

```
Condition =  
  (Age >= 18)  
  AND  
  (Age <= 30)  
  AND  
  (Category == "ECE" OR Category == "CSE")  
  AND  
  (Score >= 60)
```

```
FilteredData ← Data[Condition]
```

```
SortedData ← Sort(FilteredData, by = "Score",  
                  order = DESCENDING)
```

Indexing Technique

For large datasets, an index can be created on frequently queried columns:

```
CreateIndex(D, "Category")  
CreateIndex(D, "Age")
```

```
CandidateRows ← IndexLookup(  
    Category IN ValidCategories  
)
```

```
FilteredData ← ApplyRemainingConditions(CandidateRows)
```

This avoids unnecessarily examining every record when an indexed lookup can reduce the candidate set.

Performance Optimization for Large Datasets

1. **Filter early:** Apply the most restrictive conditions first.
2. **Use vectorized logical operations:** Avoid unnecessary row-by-row loops.
3. **Use indexing:** Index frequently searched columns such as ID, Category, or Age.
4. **Select only required columns:** Avoid loading unnecessary data.

5. **Use chunk processing:** For datasets larger than available memory, process the file in manageable chunks.
6. **Sort only after filtering:** Sorting the smaller filtered dataset reduces computational cost.
7. **Use an efficient sorting algorithm:** Efficient comparison sorting generally requires approximately $O(n \log n)$ time.
8. **Reset the index only after filtering and sorting:** This avoids unnecessary index operations.

Processing Flow

Input Dataset



Validate Columns



Create/Use Index



Remove Invalid Values



Apply Range Conditions



Apply Category Conditions



Combine Logical Conditions



Filter Dataset



Validate Constraints



Efficient Sorting



Reset Index



Save Result

Final output: A dataset containing only records satisfying all specified filtering conditions, arranged according to the required sorting order.

5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.

ANSWER:

Pseudocode Algorithm: Complete Data Analysis Workflow

Input: Data file input.csv

Output: Cleaned DataFrame, summarized DataFrame, and visualization plots

1. Pseudocode

ALGORITHM Complete_Data_Analysis

INPUT:

InputFile ← "input.csv"

SummaryFile ← "summary.csv"

OUTPUT:

SummaryData

GeneratedPlots

BEGIN

1. INITIALIZE

Import DataFrame library

Import numerical/array library

Import plotting library

2. READ DATA FROM FILE

TRY

RawData ← ReadCSV(InputFile)

CATCH FileNotFoundError

Display "Error: Input file does not exist"

STOP

CATCH PermissionError

Display "Error: File access denied"

STOP

CATCH InvalidFormatError

Display "Error: Invalid file format"

STOP

END TRY

3. VALIDATE DATA

IF RawData is empty THEN

Display "Error: Dataset is empty"

STOP

END IF

Display number of rows and columns

Check whether required columns exist

IF required columns are missing THEN

Display "Error: Required columns are missing"

STOP

END IF

4. CONVERT TO DATAFRAME

DataFrame ← ConvertToDataFrame(RawData)

Display DataFrame information

Display column names

Display data types

5. CLEAN DATA

Remove duplicate records

FOR each column C in DataFrame DO

IF C is numerical THEN

Replace missing values with Median(C)

ELSE IF C is categorical THEN

Replace missing values with Mode(C)

END IF

END FOR

Remove invalid records

6. APPLY TRANSFORMATIONS

Standardize categorical values

Convert date columns to DateTime format

Convert numerical columns to appropriate data types

Create required derived columns

Example:

$\text{TotalSales} \leftarrow \text{Quantity} \times \text{Price}$

Filter records according to analysis requirements

7. SELECT REQUIRED DATA

$\text{AnalysisData} \leftarrow \text{Select only required columns}$

8. GROUP DATA

$\text{GroupedData} \leftarrow \text{GroupBy}(\text{AnalysisData}, \text{GroupColumn})$

9. PERFORM AGGREGATION

$\text{SummaryData} \leftarrow \text{Aggregate GroupedData using:}$

Count

Sum

Mean

Minimum

Maximum

Example:

$\text{SummaryData} \leftarrow \text{GroupBy}(\text{Category})$
 $\quad .\text{Aggregate}(\text{Total} = \text{SUM}(\text{Value}), \text{Average} = \text{MEAN}(\text{Value}), \text{Count} = \text{COUNT}(\text{Value}))$

10. SORT SUMMARY

```
SummaryData ← Sort(  
    SummaryData,  
    by = Total,  
    descending = TRUE  
)
```

11. VALIDATE SUMMARY

```
IF SummaryData is empty THEN  
    Display "Error: No summarized records"  
    STOP  
END IF
```

Check for missing or invalid aggregate values

12. SAVE SUMMARY DATA

```
TRY  
    Write SummaryData to SummaryFile  
  
CATCH PermissionError  
    Display "Error: Cannot write output file"  
  
CATCH IOError  
    Display "Error while saving summary"  
  
END TRY
```

13. PREPARE DATA FOR VISUALIZATION

```
VisualizationData ← Select relevant columns
```

from SummaryData

Remove invalid visualization values

14. GENERATE BAR PLOT

Create BarPlot

X-axis $\leftarrow$ GroupColumn

Y-axis $\leftarrow$ AggregatedValue

Add title and axis labels

15. GENERATE LINE PLOT

IF Date/Time column exists THEN

Create LinePlot

X-axis $\leftarrow$ Date/Time

Y-axis $\leftarrow$ AggregatedValue

Add title and axis labels

END IF

16. GENERATE SCATTER PLOT

IF two numerical variables are available THEN

Create ScatterPlot

X-axis $\leftarrow$ Variable1

Y-axis $\leftarrow$ Variable2

Add title and axis labels

END IF

17. SAVE PLOTS

TRY

Save BarPlot as "bar_plot.png"

IF LinePlot exists THEN

Save LinePlot as "line_plot.png"

END IF

IF ScatterPlot exists THEN

Save ScatterPlot as "scatter_plot.png"

END IF

CATCH IOError

Display "Warning: Plot could not be saved"

END TRY

18. DISPLAY FINAL OUTPUT

Display cleaned DataFrame information

Display SummaryData

Display generated plots

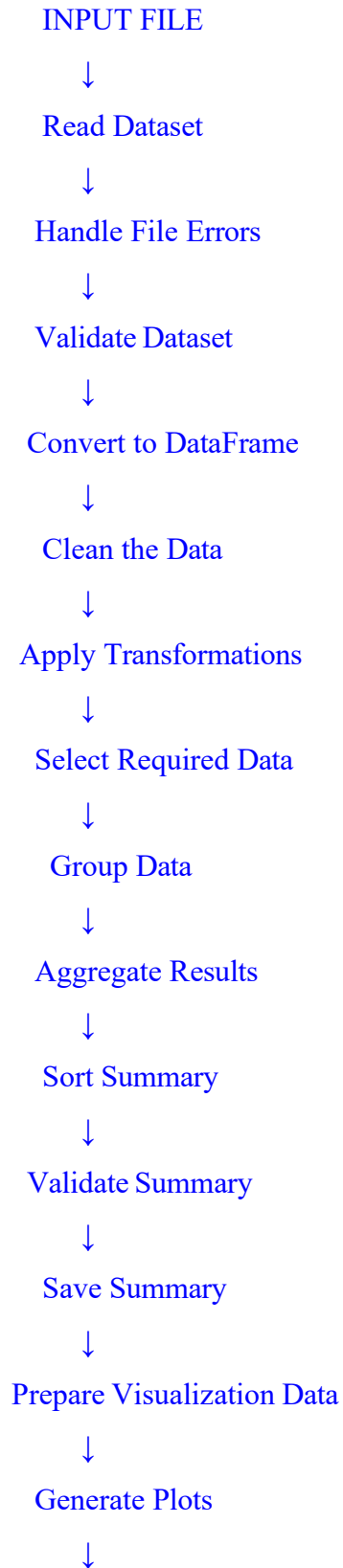
Display "Data analysis completed successfully"

19. RETURN RESULTS

RETURN SummaryData, GeneratedPlots

END

2. Workflow



Save Plot Files



FINAL OUTPUT

3. Example of Grouping and Aggregation

For a student dataset containing Department and Marks:

```
SummaryData ← DataFrame
    .GroupBy("Department")
    .Aggregate(
        StudentCount = COUNT(Student_ID),
        AverageMarks = MEAN(Marks),
        MaximumMarks = MAX(Marks),
        MinimumMarks = MIN(Marks)
    )
```

The summarized result can then be used to generate a bar chart comparing average marks by department.

4. Performance Optimization

- Use vectorized DataFrame operations instead of unnecessary row-by-row processing.
- Remove invalid and duplicate records before grouping.
- Select only required columns to reduce memory usage.
- Use built-in GroupBy and aggregation functions.
- For very large files, read and process the data in chunks.
- Generate plots from the summarized data rather than the complete raw dataset.
- Save intermediate summarized results to avoid repeating expensive calculations.

Final result: The algorithm produces a cleaned dataset → grouped and aggregated summary → validated output file → visualization plots, while also handling file errors and invalid data at each stage.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

